Graphs and Their Code

Why Buses Bunch · ELE IDSA Project 1 · GEC2B · Group 10

We made **8 graphs**. Only **3 (graphs 6–8)** test the link between passengers and bunching; the rest are background.

Which graphs are which

Type	Graphs	What they do
Data check	1	Shows the cleaning worked. Not a result.
Ridership (demand only)	2, 3, 4	Where and when people board. No bunching yet.
Bunching (spacing only)	5	Proves bunching exists. No passengers yet.
Hypothesis test	6, 7, 8	Connect passengers to bunching (H1–H3).

Heatmap (graph 2) in one line: "This doesn't measure bunching yet. It shows where demand piles up. H2 later shows that boardings slow the bus, so these hot spots are where delays most likely start."

Overview

#	Graph	Type	Slide	Made by
1	**Dwell time before and after cleaning** fig01_dwell_cleaning.png	Data check	slide 7	Rence
2	**Average weekday boardings by stop and hour** fig02_boardings_heatmap.png	Ridership (demand only)	slide 10	Rence
3	**Boardings by hour, weekdays vs weekends** fig03_boardings_by_hour.png	Ridership (demand only)	slide 8	Elijah
4	**Total boardings per day, Jul–Dec 2021** fig04_daily_boardings.png	Ridership (demand only)	slide 9	Elijah
5	**Distribution of headways, weekdays 7:00–17:59** fig05_headway_distribution.png	Bunching (spacing only)	slide 11	Elijah
6	**H1: Passengers on board by headway group** fig06_h1_load_vs_headway.png	Hypothesis test (demand ↔ bunching)	slide 12	Khalil
7	**H2: Dwell time vs passengers boarding** fig07_h2_dwell_vs_boardings.png	Hypothesis test (demand ↔ bunching)	slide 13	Khalil
8	**H3: Headway irregularity along the route**	Hypothesis test (demand ↔ bunching)	slide 14	Khalil

fig08_h3_headway_cv_by_stop.png

How to run the code

- Open the notebooks in `notebooks/` and run top to bottom. The setup cell loads data; each graph cell saves a PNG to `figures/`.
- `new_fig()`, `finish()`, `save()` (in `src/project.py`) give every graph the same style.

Shared setup (run first in each notebook)

notebooks/01_data_preparation.ipynb (Rence) · graphs 1–2

```
import sys; sys.path.insert(0, '../src')
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from project import load_events, load_stops, new_fig, finish, save, COLORS, SEQ_CMAP, SOURCE_NOTE

ev = load_events()
stops = load_stops()
print(ev.shape)
ev.head()
```

notebooks/02_ridership_patterns.ipynb (Elijah) · graphs 3–5

```
import sys, warnings
warnings.filterwarnings('ignore')
sys.path.insert(0, '../src')
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.dates import MonthLocator, DateFormatter
from project import load_events, window, new_fig, finish, save, COLORS, SOURCE_NOTE

ev = load_events()
w = window(ev)
# Service day 2021-06-30 holds only 8 records (trips just after midnight), so it is left out of per-day
averages.
days = ev[ev.service_day > '2021-06-30']
print(f'ev: {len(ev):,} rows  |  window: {len(w):,} rows  |  service days used:
{days.service_day.nunique()}')
```

notebooks/03_hypotheses.ipynb (Khalil) · graphs 6–8

```
import sys
sys.path.insert(0, "../src")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf
from scipy import stats

from project import (load_events, window, new_fig, finish, save,
                     COLORS, SERIES, SOURCE_NOTE)

ALPHA = 0.05


# Format a p-value; SciPy returns exactly 0.0 when the p-value underflows.
def pfmt(p):
    return f"= {p:.3g}" if p > 0 else "< 1e-308 (underflow)"


ev = load_events()
w = window(ev)

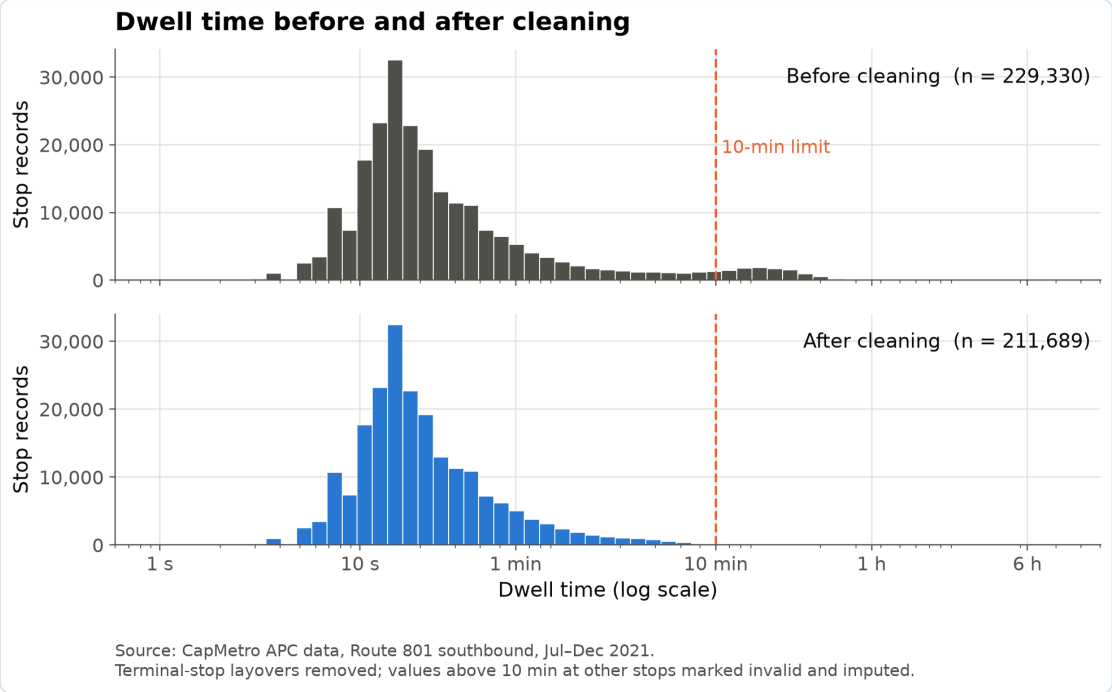
print(f"all stop events        : {len(ev):,}")
print(f"analysis window (w)    : {len(w):,}")
print(f"service days in window : {w.service_day.nunique()}")
print(f"alpha                  : {ALPHA}")
```

1 Dwell time before and after cleaning

Rence

Chart type: Two histograms (log x-axis) · **File:** figures/fig01_dwell_cleaning.png · **Slide** 7

> Data check **Link to bunching:** None. It only proves the data is clean enough to trust.



What it shows

- **Before:** most stops last 10–60 s, but a tail runs past 10 minutes (end-stop layovers and errors).
- **Why cut at 10 min:** doors open that long is a parked or held bus, not boarding (20 boardings ≈ 112 s).
- **After:** layovers removed; 381 bad values imputed (stop × hour median). Main peak unchanged.

How to read: Log x-axis (1 s to 1 h). Orange dashed line = 10-minute limit.

Python code

notebooks/01_data_preparation.ipynb, cell 7 (after the shared setup)

```
bins = np.logspace(0, np.log10(30000), 60)
before = ev.dwell_time_raw[ev.dwell_time_raw > 0]
after = ev.dwell_time.dropna()
after = after[after > 0]

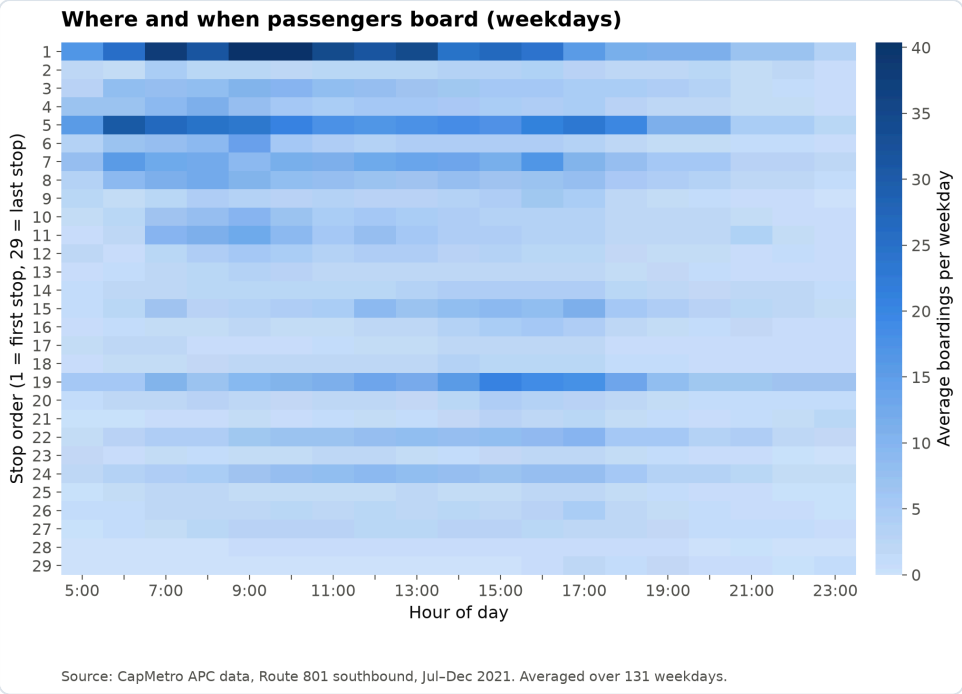
fig, axes = plt.subplots(2, 1, figsize=(10, 6.4), sharex=True)
for ax, data, color, label in [(axes[0], before, COLORS['muted'], f'Before cleaning  (n =
{len(before):,})'),
                               (axes[1], after, COLORS['blue'], f'After cleaning  (n = {len(after):,})')]:
    ax.hist(data, bins=bins, color=color, edgecolor='white', linewidth=0.6)
    ax.set_xscale('log')
    ax.set_ylabel('Stop records')
    ax.text(0.99, 0.92, label, transform=ax.transAxes, ha='right', va='top', fontsize=12.5,
color=COLORS['text'])
    ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f'{v:,.0f}'))
for ax in axes:
    ax.axvline(600, color=COLORS['orange'], linewidth=1.5, linestyle='--')
axes[0].text(640, axes[0].get_ylim()[1] * 0.55, '10-min limit', color=COLORS['orange'], fontsize=11.5)
ticks = [1, 10, 60, 600, 3600, 21600]
axes[1].set_xticks(ticks, ['1 s', '10 s', '1 min', '10 min', '1 h', '6 h'])
axes[0].set_title('Dwell time before and after cleaning')
axes[1].set_xlabel('Dwell time (log scale)')
axes[1].annotate(SOURCE_NOTE + '.\nTerminal-stop layovers removed; values above 10 min at other stops marked
invalid and imputed.',
                 xy=(0, -0.42), xycoords='axes fraction', fontsize=10.5, color=COLORS['muted'], va='top')
fig.tight_layout()
save(fig, 'fig01_dwell_cleaning');
```

2 Average weekday boardings by stop and hour

Rence

Chart type: Heatmap · **File:** figures/fig02_boardings_heatmap.png · **Slide** 10

Ridership (demand only) **Link to bunching:** Doesn't measure bunching. Shows **where** demand piles up; with H2 (6.9 s per boarding), these are the stops where buses lose the most time.



What it shows

- Busiest stops: **1** (~442/day), **5** (~329, mornings), **19** (~204, afternoons).
- Stops after 20 are pale: mostly alighting.

How to read: Rows = stops in route order, columns = hours. Darker = more boardings (131-weekday average).

Python code

notebooks/01_data_preparation.ipynb, cells 9, 10 (after the shared setup)

```
# drop service day 2021-06-30: it only holds 8 records (trips just after midnight), not a full day
wk = ev[(ev.day_type == 'Weekday') & (ev.service_day > '2021-06-30')]
n_days = wk.service_day.nunique()
grid = (wk.groupby(['stop_order', 'hour']).ons.sum() / n_days).unstack(fill_value=0)
grid = grid.loc[:, 5:23]   # service hours with meaningful ridership

fig, ax = plt.subplots(figsize=(11, 7.5))
im = ax.imshow(grid.values, aspect='auto', cmap=SEQ_CMAP, interpolation='nearest')
ax.set_xticks(range(len(grid.columns)), [f'{h}:00' if h % 2 == 1 else '' for h in grid.columns])
ax.set_yticks(range(len(grid.index)), grid.index)
ax.grid(False)
for s in ax.spines.values():
    s.set_visible(False)
cb = fig.colorbar(im, ax=ax, pad=0.02)
cb.set_label('Average boardings per weekday')
cb.outline.set_visible(False)
finish(ax, 'Where and when passengers board (weekdays)', 'Hour of day',
       'Stop order (1 = first stop, 29 = last stop)',
       note=SOURCE_NOTE + f'. Averaged over {n_days} weekdays.')
fig.tight_layout()
save(fig, 'fig02_boardings_heatmap');

top = ((wk.groupby('stop_order').ons.sum() / n_days).sort_values(ascending=False).head(5).rename('avg
weekday boardings').round(0)
       .to_frame().join(stops.set_index('stop_order')[['stop_id']]))
```

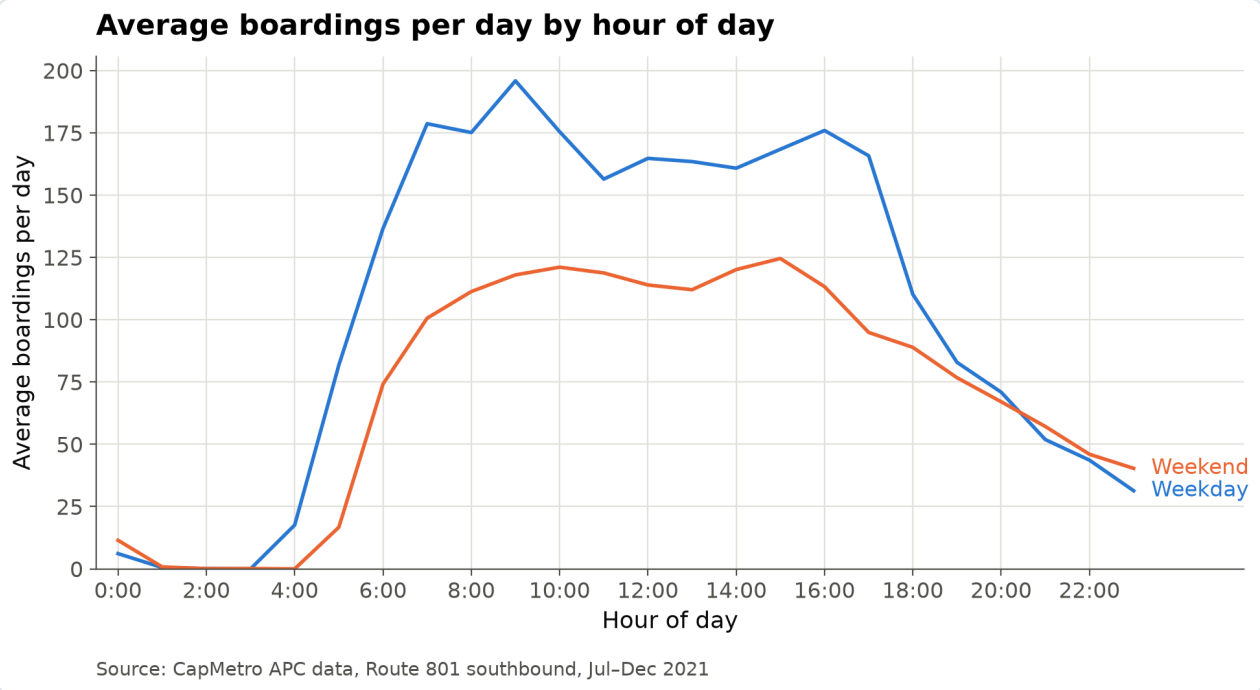
```
print(f'Busiest stops, whole-day boardings per weekday ({n_days} weekdays):'); print(top)
print('Busiest hour:', grid.sum(axis=0).idxmax(), ':00')
```

3 Boardings by hour, weekdays vs weekends

Elijah

Chart type: Line chart · **File:** figures/fig03_boardings_by_hour.png · **Slide** 8

Ridership (demand only) **Link to bunching:** Shows **when** demand is highest, which is why the analysis uses weekdays 7:00–17:59.



What it shows

- Weekday peaks at **9:00 (195.9)** and **16:00 (175.9)**; weekends flatter, peak 15:00 (124.6).
- **2,514** boardings per weekday vs **1,728** per weekend day.

How to read: Average boardings per hour of day; lines labelled at the right end.

Python code

notebooks/02_ridership_patterns.ipynb, cell 4 (after the shared setup)

```
n_days = days.groupby('day_type').service_day.nunique()
bph = (days.groupby(['day_type', 'hour']).ons.sum() / n_days).rename('avg_boardings').reset_index()
wkday = bph[bph.day_type == 'Weekday'].set_index('hour').avg_boardings.reindex(range(24), fill_value=0)
wkend = bph[bph.day_type == 'Weekend'].set_index('hour').avg_boardings.reindex(range(24), fill_value=0)

daily_tot = days.groupby(['service_day', 'day_type']).ons.sum().reset_index()
wd_avg = daily_tot[daily_tot.day_type == 'Weekday'].ons.mean()
we_avg = daily_tot[daily_tot.day_type == 'Weekend'].ons.mean()
pm = wkday.loc[12:23]
print(f'Days: {n_days.to_dict()}')
print(f'Weekday peak: hour {wkday.idxmax()}, {wkday.max():.1f} avg boardings')
print(f'Weekday afternoon peak: hour {pm.idxmax()}, {pm.max():.1f} avg boardings')
print(f'Weekend peak: hour {wkend.idxmax()}, {wkend.max():.1f} avg boardings '
      f'({(1 - wkend.max() / wkday.max()) * 100:.0f}% below the weekday peak)')
print(f'Weekday avg boardings/day: {wd_avg:.0f}  |  Weekend: {we_avg:.0f}  |  weekday is {(wd_avg / we_avg -
1) * 100:.0f}% higher')

fig, ax = new_fig(10, 5.6)
ax.plot(wkday.index, wkday.values, color=COLORS['blue'])
ax.plot(wkend.index, wkend.values, color=COLORS['orange'])
# direct labels at the right end, pushed apart so they never overlap
y_wd, y_we = wkday.iloc[-1], wkend.iloc[-1]
gap = 9
if abs(y_wd - y_we) < gap:
    mid = (y_wd + y_we) / 2
    y_wd, y_we = (mid - gap / 2, mid + gap / 2) if y_wd < y_we else (mid + gap / 2, mid - gap / 2)
ax.text(23.4, y_wd, 'Weekday', color=COLORS['blue'], va='center', fontsize=12)
ax.text(23.4, y_we, 'Weekend', color=COLORS['orange'], va='center', fontsize=12)
ax.set_xticks(range(0, 24, 2), [f'{h}:00' for h in range(0, 24, 2)])
```

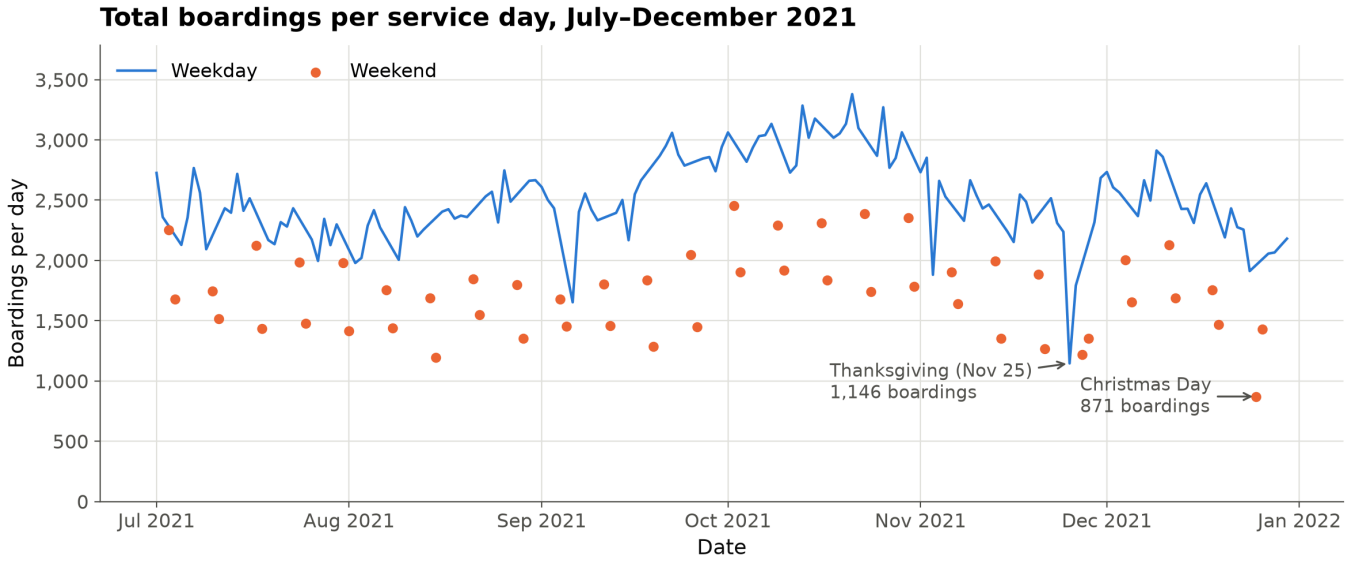
```
ax.set_xlim(-0.5, 25.5)
ax.set_ylim(0, None)
finish(ax, 'Average boardings per day by hour of day', 'Hour of day', 'Average boardings per day',
       note=SOURCE_NOTE)
fig.tight_layout()
save(fig, 'fig03_boardings_by_hour')
plt.show()
```

4 Total boardings per day, Jul–Dec 2021

Elijah

Chart type: Time series · **File:** figures/fig04_daily_boardings.png · **Slide** 9

Ridership (demand only) **Link to bunching:** Context only: the six-month trend, no link to bunching.



What it shows

- Weekdays stay above weekends; peak month October (~3,025/weekday).
- Holiday dips: Thanksgiving 1,146; Christmas 871 (lowest).

How to read: Each point = one day's total boardings (blue line weekdays, orange dots weekends).

Python code

notebooks/02_ridership_patterns.ipynb, cell 6 (after the shared setup)

```
daily_wd = daily_tot[daily_tot.day_type == 'Weekday'].sort_values('service_day')
daily_we = daily_tot[daily_tot.day_type == 'Weekend'].sort_values('service_day')
std_wd = daily_wd.ons.std()
low_wd = daily_wd.loc[daily_wd.ons.idxmin()]
low_we = daily_we.loc[daily_we.ons.idxmin()]
monthly = daily_wd.groupby(daily_wd.service_day.dt.strftime('%b')).ons.mean().reindex(
    ['Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']).round(0)
print(f'Overall avg boardings/day: {daily_tot.ons.mean():.0f}')
print(f'Weekday avg: {wd_avg:.0f} (sd {std_wd:.0f})  |  Weekend avg: {we_avg:.0f}')
print(f'Lowest weekday: {low_wd.ons:.0f} on {low_wd.service_day.date()} '
      f'({(wd_avg - low_wd.ons) / std_wd:.1f} sd below the weekday mean)')
print(f'Lowest weekend day: {low_we.ons:.0f} on {low_we.service_day.date()}')
print('Average weekday boardings by month:', monthly.to_dict())

fig, ax = new_fig(12, 5.6)
ax.plot(daily_wd.service_day, daily_wd.ons, color=COLORS['blue'], linewidth=1.6, label='Weekday')
ax.scatter(daily_we.service_day, daily_we.ons, color=COLORS['orange'], s=28, label='Weekend', zorder=3)
ax.annotate(f'Thanksgiving ({low_wd.service_day:%b %d})\n{low_wd.ons:,.0f} boardings',
            xy=(low_wd.service_day, low_wd.ons), xytext=(-150, -12), textcoords='offset points',
            arrowprops=dict(arrowstyle='->', color=COLORS['muted'], lw=1.2),
            fontsize=11, color=COLORS['muted'], va='center')
ax.annotate(f'Christmas Day\n{low_we.ons:,.0f} boardings',
            xy=(low_we.service_day, low_we.ons), xytext=(-110, 0), textcoords='offset points',
            arrowprops=dict(arrowstyle='->', color=COLORS['muted'], lw=1.2),
            fontsize=11, color=COLORS['muted'], va='center')
ax.legend(loc='upper left', ncol=2)
ax.set_ylim(0, daily_tot.ons.max() * 1.12)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f'{v:,.0f}'))
ax.xaxis.set_major_locator(MonthLocator())
ax.xaxis.set_major_formatter(DateFormatter('%b %Y'))
finish(ax, 'Total boardings per service day, July–December 2021', 'Date', 'Boardings per day',
       note=SOURCE_NOTE)
```

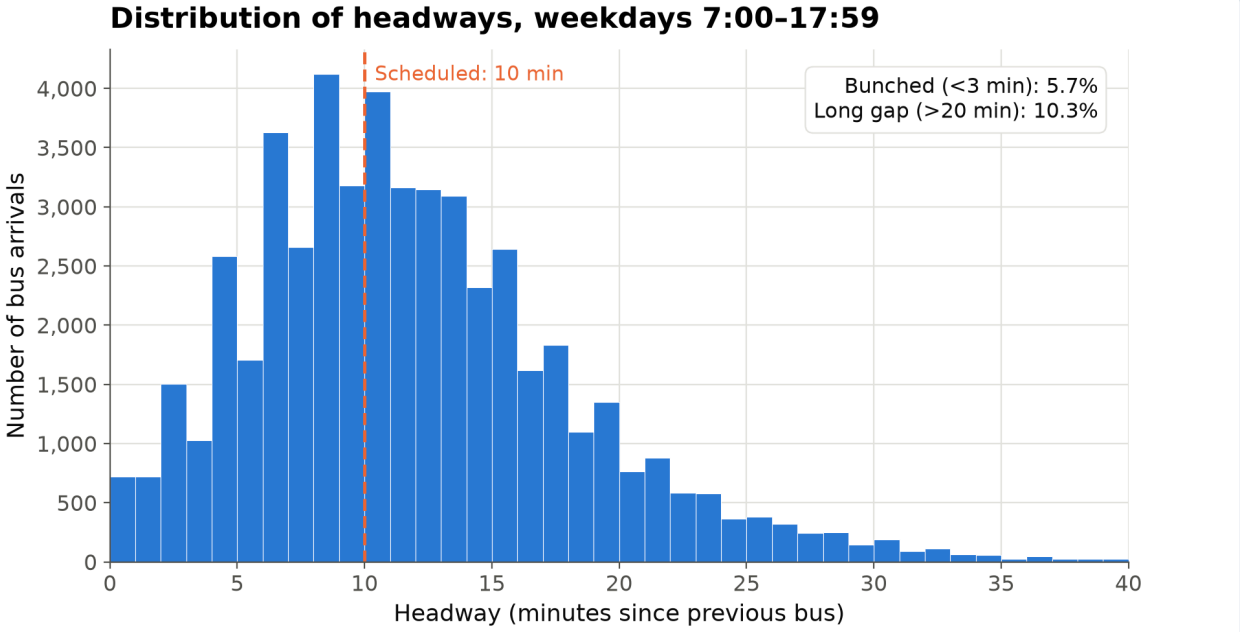
```
fig.tight_layout()
save(fig, 'fig04_daily_boardings')
plt.show()
```

5 Distribution of headways, weekdays 7:00–17:59

Elijah

Chart type: Histogram · **File:** figures/fig05_headway_distribution.png · **Slide** 11

Bunching (spacing only) **Link to bunching:** Proves bunching **exists**: buses often don't arrive 10 min apart.



What it shows

- 51,437 headways, median **11.0 min** vs the 10-min schedule.
- **5.7%** under 3 min (bunched), **10.3%** over 20 min (long gaps).

How to read: Bars = arrivals per 1-minute gap bin. Dashed line = 10-min schedule.

Python code

notebooks/02_ridership_patterns.ipynb, cell 8 (after the shared setup)

```
hw = (w.headway_s / 60).dropna()
pct_bunched = (hw < 3).mean() * 100
pct_long = (hw > 20).mean() * 100
counts = pd.cut(hw, range(0, 61), right=False).value_counts()
mode_bin = counts.idxmax()
over40 = (hw >= 40).sum()
print(f'Valid headways: {len(hw):,}  |  median {hw.median():.1f} min')
print(f'Bunched (<3 min): {pct_bunched:.1f}%  |  Long gap (>20 min): {pct_long:.1f}%')
print(f'Most common 1-min bin: {mode_bin.left}-{mode_bin.right} min ({counts.max():,} arrivals); '
      f'next: {counts.sort_values(ascending=False).index[1]}')
print(f'Headways of 40 min or more (not drawn): {over40:,} ({over40 / len(hw) * 100:.1f}%)')

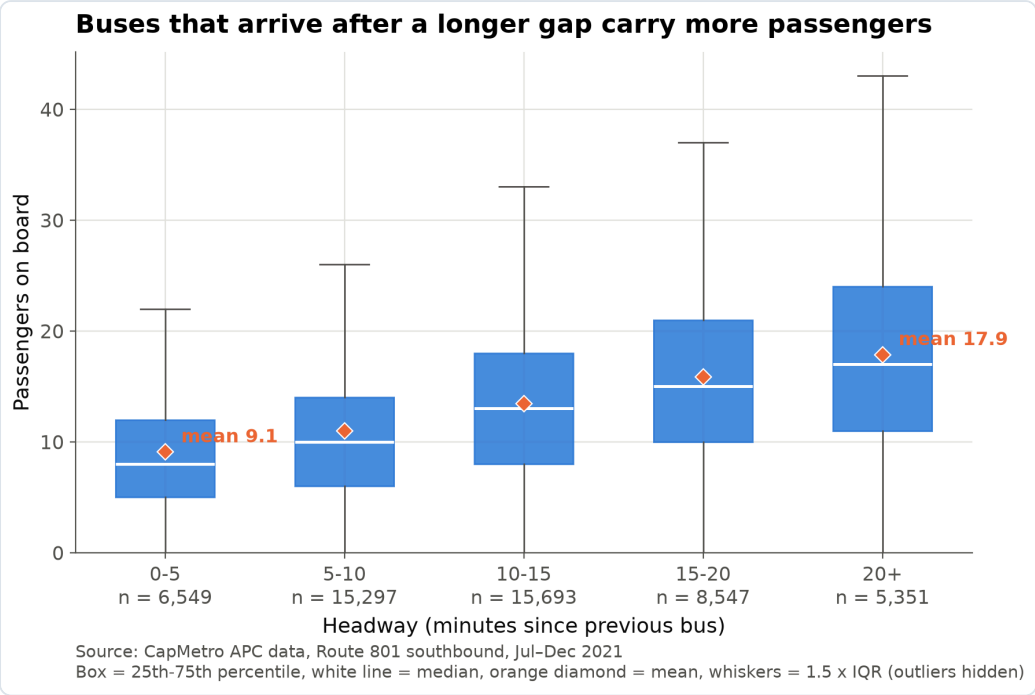
fig, ax = new_fig(10, 5.6)
ax.hist(hw[hw < 40], bins=range(0, 41), color=COLORS['blue'], edgecolor='white', linewidth=0.4)
ax.axvline(10, color=COLORS['orange'], linestyle='--', linewidth=1.8)
ax.text(10.4, ax.get_ylim()[1] * 0.97, 'Scheduled: 10 min', color=COLORS['orange'], fontsize=11, va='top')
ax.text(0.97, 0.95, f'Bunched (<3 min): {pct_bunched:.1f}%\nLong gap (>20 min): {pct_long:.1f}%',
        transform=ax.transAxes, fontsize=11.5, va='top', ha='right', color=COLORS['text'],
        bbox=dict(boxstyle='round,pad=0.4', facecolor='white', edgecolor=COLORS['grid'], linewidth=0.8))
ax.set_xlim(0, 40)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f'{v:,.0f}'))
finish(ax, 'Distribution of headways, weekdays 7:00–17:59', 'Headway (minutes since previous bus)',
       'Number of bus arrivals',
       note=SOURCE_NOTE + f'. {len(hw):,} headways; {over40:,} of 40 min or more not shown.')
fig.tight_layout()
save(fig, 'fig05_headway_distribution')
plt.show()
```

6 H1: Passengers on board by headway group

Khalil

Chart type: Box plot · **File:** figures/fig06_h1_load_vs_headway.png · **Slide** 12

Hypothesis test (demand ↔ bunching) **Link to bunching:** Bigger gap → more passengers on board.



What it shows

- Mean load **9.1** (0–5 min gap) → **17.9** (20+ min).
- Spearman ρ = 0.349, Kruskal–Wallis p < 0.001: **reject H0**.

How to read: Box = middle 50%, white line = median, orange diamond = mean.

Python code

notebooks/03_hypotheses.ipynb, cells 3, 4, 5 (after the shared setup)

```
h1 = w[w.headway_s.notna()].copy()
h1["headway_min"] = h1.headway_s / 60

EDGES = [0, 5, 10, 15, 20, np.inf]
LABELS = ["0-5", "5-10", "10-15", "15-20", "20+"]
h1["headway_group"] = pd.cut(h1.headway_min, bins=EDGES, labels=LABELS, right=False)

g = h1.groupby("headway_group", observed=True).max_load
h1_stats = pd.DataFrame({
    "n": g.size(),
    "mean_load": g.mean().round(2),
    "median_load": g.median(),
    "q25": g.quantile(0.25),
    "q75": g.quantile(0.75),
})

print(f"rows with a valid headway: {len(h1):,}")
print(f"headway: median {h1.headway_min.median():.1f} min, mean {h1.headway_min.mean():.1f} min")
print()
print("Mean passengers on board by headway group")
print(h1_stats.to_string())

# --- Test 1: Spearman rank correlation (headway vs load) ---
rho, p_rho = stats.spearmanr(h1.headway_s, h1.max_load)

# --- Test 2: Kruskal-Wallis across the five headway groups ---
groups = [grp.max_load.values for _, grp in h1.groupby("headway_group", observed=True)]
H, p_kw = stats.kruskal(*groups)
```

```
print("H1 test results")
print(f"  Spearman rho    = {rho:.3f}   p {pfmt(p_rho)}   (n = {len(h1):,})")
print(f"  Kruskal-Wallis H = {H:.1f}   df = {len(groups) - 1}   p {pfmt(p_kw)}")
print(f"  mean load: {h1_stats.mean_load.iloc[0]:.1f} passengers in the 0-5 min group "
      f"-> {h1_stats.mean_load.iloc[-1]:.1f} in the 20+ min group")
print(f"  reject H0 (Spearman):       {p_rho < ALPHA}")
print(f"  reject H0 (Kruskal-Wallis): {p_kw < ALPHA}")

fig, ax = new_fig()

data = [h1.loc[h1.headway_group == lab, "max_load"].values for lab in LABELS]
bp = ax.boxplot(
    data, showfliers=False, widths=0.55, patch_artist=True, showmeans=True,
    medianprops=dict(color="white", linewidth=1.8),
    boxprops=dict(facecolor=COLORS["blue"], edgecolor=COLORS["blue"], alpha=0.85),
    whiskerprops=dict(color=COLORS["muted"]),
    capprops=dict(color=COLORS["muted"]),
    meanprops=dict(marker="D", markersize=7, markerfacecolor=COLORS["orange"],
                   markeredgecolor="white", markeredgewidth=0.8),
)

# n under each box label
ax.set_xticks(range(1, len(LABELS) + 1))
ax.set_xticklabels([f"{lab}\nn = {h1_stats.n[lab]:,}" for lab in LABELS])

# label only the first and last group mean
for i in (0, len(LABELS) - 1):
    m = h1_stats.mean_load.iloc[i]
    ax.annotate(f"mean {m:.1f}", xy=(i + 1, m), xytext=(10, 6), textcoords="offset points",
                color=COLORS["orange"], fontsize=11.5, fontweight="bold")

ax.set_ylim(0, None)

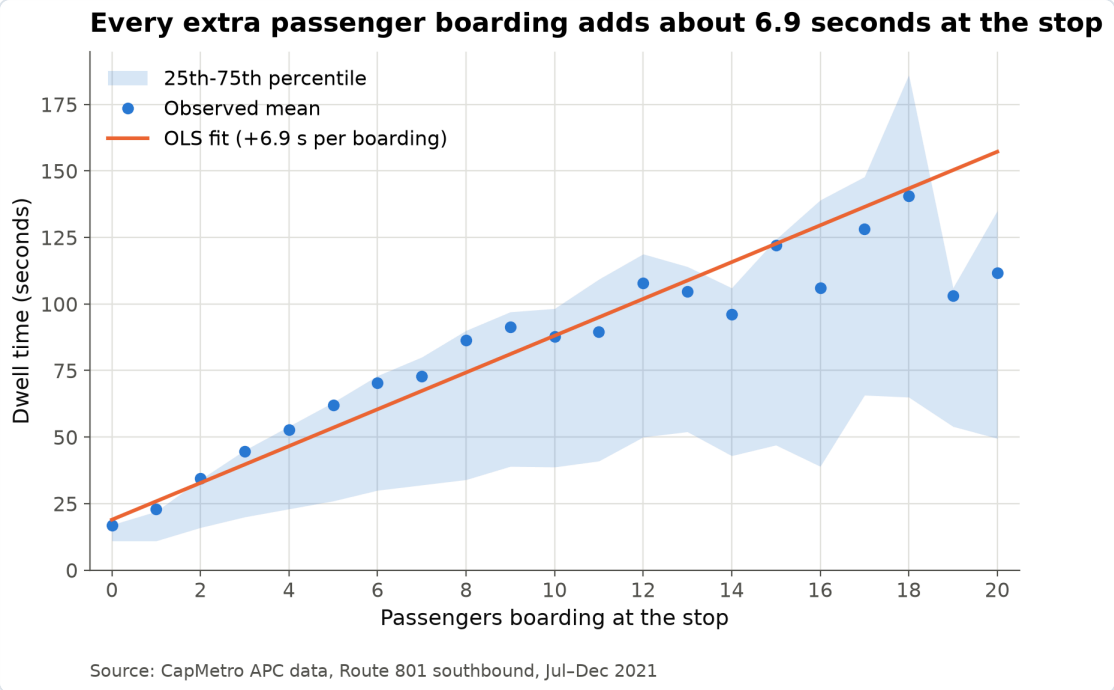
finish(ax,
       "Buses that arrive after a longer gap carry more passengers",
       "Headway (minutes since previous bus)",
       "Passengers on board",
       note=SOURCE_NOTE + "\n" + "Box = 25th-75th percentile, white line = median, "
            "orange diamond = mean, whiskers = 1.5 x IQR (outliers hidden)")
print(save(fig, "fig06_h1_load_vs_headway"))
plt.show()
```

7 H2: Dwell time vs passengers boarding

Khalil

Chart type: Scatter + regression line · **File:** figures/fig07_h2_dwell_vs_boardings.png · **Slide** 13

Hypothesis test (demand ↔ bunching) **Link to bunching:** More passengers → longer stops → bus falls behind.



What it shows

- Each boarding adds **6.9 s**, each alighting **3.7 s** (OLS, n = 116,697).
- $p < 0.001$: **reject H0**; $R^2 = 0.18$.

How to read: Dots = mean dwell per number of boardings; band = middle 50%; orange line = regression fit.

Python code

notebooks/03_hypotheses.ipynb, cells 7, 8, 9 (after the shared setup)

```
h2 = w[w.is_terminal == False].copy()

model = smf.ols("dwell_time ~ ons + offs", data=h2).fit()

print(f"rows used: {len(h2):,} non-terminal stop events")
print()
print(model.summary().tables[1])
print()
print(f"R-squared          = {model.rsquared:.3f}")
print(f"adj. R-squared     = {model.rsquared_adj:.3f}")
print(f"F({int(model.df_model)}, {int(model.df_resid)}) = {model.fvalue:.1f}   p {pfmt(model.f_pvalue)}")
print()
for name in ("Intercept", "ons", "offs"):
    print(f"  {name:<10} b = {model.params[name]:.3f} s   "
          f"t = {model.tvalues[name]:.1f}   p {pfmt(model.pvalues[name])}   "
          f"95% CI [{model.conf_int().loc[name, 0]:.3f}, {model.conf_int().loc[name, 1]:.3f}]")
print()
print(f"  reject H0 for ons : {model.pvalues['ons'] < ALPHA}")
print(f"  reject H0 for offs: {model.pvalues['offs'] < ALPHA}")
print(f"  mean dwell time   : {h2.dwell_time.mean():.1f} s (median {h2.dwell_time.median():.1f} s)")
print(f"  mean offs         : {h2.offs.mean():.2f} passengers")

ONS_MAX = 20
prof = (h2[h2.ons <= ONS_MAX]
        .groupby("ons")
        .dwell_time
        .agg(n="size", mean="mean", q25=lambda s: s.quantile(0.25), q75=lambda s: s.quantile(0.75))
        .reset_index())
```

```
offs_mean = h2.offs.mean()
xx = np.arange(0, ONS_MAX + 1)
fit = model.params["Intercept"] + model.params["ons"] * xx + model.params["offs"] * offs_mean

print("Observed mean dwell time by boardings (ons)")
print(prof.round(2).to_string(index=False))
print()
print(f"fitted line (offs held at its mean of {offs_mean:.2f}): "
      f"dwell = {model.params['Intercept'] + model.params['offs'] * offs_mean:.1f} "
      f"+ {model.params['ons']:.2f} x ons")
print(f"  0 boarding  -> fitted {fit[0]:.1f} s (observed mean {prof.loc[prof.ons == 0, 'mean'].iloc[0]:.1f}
s)")
print(f"  20 boarding -> fitted {fit[-1]:.1f} s (observed mean {prof.loc[prof.ons == ONS_MAX,
'mean'].iloc[0]:.1f} s)")

fig, ax = new_fig()

ax.fill_between(prof.ons, prof.q25, prof.q75, color=COLORS["blue"], alpha=0.18,
                linewidth=0, label="25th-75th percentile")
ax.plot(prof.ons, prof["mean"], "o", color=COLORS["blue"], markersize=6,
        label="Observed mean")
ax.plot(xx, fit, "-", color=COLORS["orange"], linewidth=2.2,
        label=f"OLS fit (+{model.params['ons']:.1f} s per boarding)")

ax.legend(loc="upper left")
ax.set_xticks(range(0, ONS_MAX + 1, 2))
ax.set_xlim(-0.5, ONS_MAX + 0.5)
ax.set_ylim(0, None)

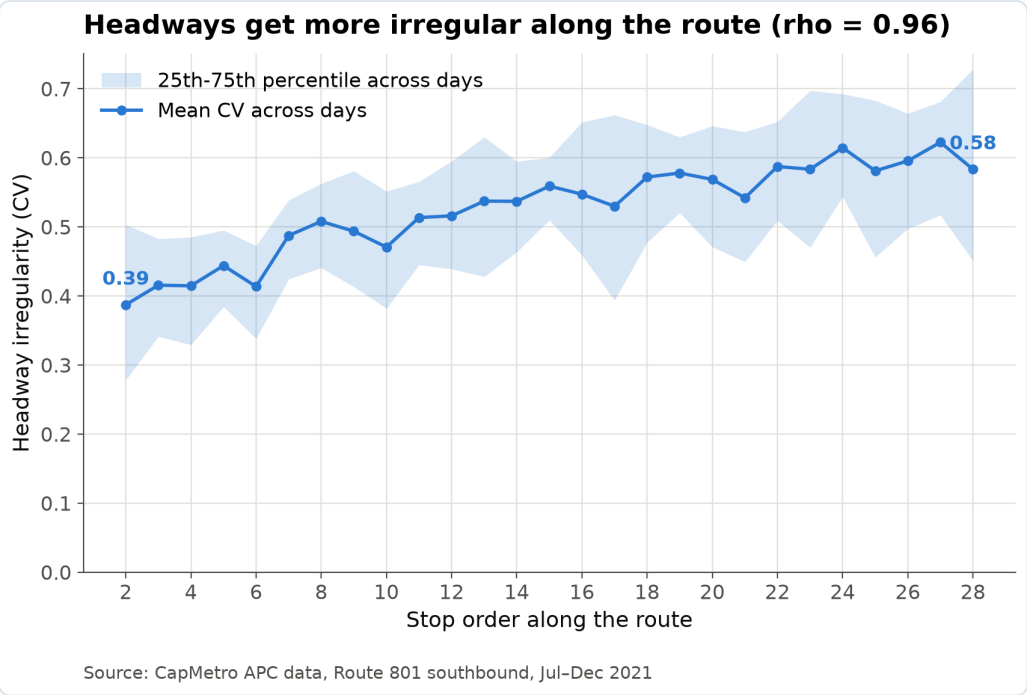
finish(ax,
       "Every extra passenger boarding adds about "
       f"{model.params['ons']:.1f} seconds at the stop",
       "Passengers boarding at the stop",
       "Dwell time (seconds)",
       note=SOURCE_NOTE)
print(save(fig, "fig07_h2_dwell_vs_boardings"))
plt.show()
```

8 H3: Headway irregularity along the route

Khalil

Chart type: Line chart + band · **File:** figures/fig08_h3_headway_cv_by_stop.png · **Slide** 14

Hypothesis test (demand ↔ bunching) **Link to bunching:** Spacing gets worse the further buses travel: bunching builds up.



What it shows

- CV (std ÷ mean headway) rises **0.39 → 0.58** from stop 2 to 28 (+51%).
- Spearman ρ = 0.955, p < 0.001: **reject H0**.

How to read: Dots = average CV per stop; band = spread across days.

Python code

notebooks/03_hypotheses.ipynb, cells 11, 12, 13 (after the shared setup)

```
h3 = w[w.headway_s.notna()].copy()

daily = (h3.groupby(["service_day", "stop_order"])
           .headway_s
           .agg(n="size", mean="mean", std="std")
           .reset_index())
daily = daily[daily.n >= 3].copy()
daily["cv"] = daily["std"] / daily["mean"]

by_stop = (daily.groupby("stop_order")
                .cv
                .agg(days="size", mean_cv="mean",
                     q25=lambda s: s.quantile(0.25), q75=lambda s: s.quantile(0.75))
                .reset_index())
by_stop = by_stop[(by_stop.stop_order >= 2) & (by_stop.stop_order <= 28)]

print(f"day x stop cells with >= 3 headways: {len(daily):,}")
print(f"stops covered: {by_stop.stop_order.min()}-{by_stop.stop_order.max()} "
      f"({len(by_stop)} stops), {daily.service_day.nunique()} service days")
print()
print(by_stop.round(3).to_string(index=False))

rho3, p3 = stats.spearmanr(by_stop.stop_order, by_stop.mean_cv)

first_cv = by_stop.mean_cv.iloc[0]
last_cv = by_stop.mean_cv.iloc[-1]
peak = by_stop.loc[by_stop.mean_cv.idxmax()]

print("H3 test results")
```

```
print(f"  Spearman rho = {rho3:.3f}   p {pfmt(p3)}   (n = {len(by_stop)} stops)")
print(f"  mean CV: {first_cv:.3f} at stop {int(by_stop.stop_order.iloc[0])} "
      f"-> {last_cv:.3f} at stop {int(by_stop.stop_order.iloc[-1])} "
      f"(+{100 * (last_cv / first_cv - 1):.0f}%)")
print(f"  highest mean CV: {peak.mean_cv:.3f} at stop {int(peak.stop_order)}")
print(f"  reject H0: {p3 < ALPHA}")

fig, ax = new_fig()

ax.fill_between(by_stop.stop_order, by_stop.q25, by_stop.q75, color=COLORS["blue"],
                alpha=0.18, linewidth=0, label="25th-75th percentile across days")
ax.plot(by_stop.stop_order, by_stop.mean_cv, "-o", color=COLORS["blue"], markersize=5,
        label="Mean CV across days")

for x, y, txt in ((by_stop.stop_order.iloc[0], first_cv, f"{first_cv:.2f}"),
                  (by_stop.stop_order.iloc[-1], last_cv, f"{last_cv:.2f}")):
    ax.annotate(txt, xy=(x, y), xytext=(0, 12), textcoords="offset points",
                ha="center", color=COLORS["blue"], fontsize=11.5, fontweight="bold")

ax.legend(loc="upper left")
ax.set_xticks(range(2, 29, 2))
ax.set_ylim(0, None)

finish(ax,
       f"Headways get more irregular along the route (rho = {rho3:.2f})",
       "Stop order along the route",
       "Headway irregularity (CV)",
       note=SOURCE_NOTE)
print(save(fig, "fig08_h3_headway_cv_by_stop"))
plt.show()
```

Appendix: shared chart style (src/project.py)

Every graph imports these helpers, so all 8 share the same fonts, colours (blue, orange, aqua), grid, title placement and source note.

```python
"""Shared helpers: load the processed data and draw every chart in one consistent style.

In a notebook (inside notebooks/):
    import sys; sys.path.insert(0, "../src")
    from project import load_events, load_stops, window, new_fig, finish, save, COLORS, SEQ_CMAP
"""
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
from matplotlib.colors import LinearSegmentedColormap

ROOT = Path(__file__).resolve().parents[1]
PROCESSED = ROOT / "data" / "processed"
FIGURES = ROOT / "figures"

# ---------------------------------------------------------------- data
def load_events(path=None):
    """One row per bus per stop (cleaned and imputed). See CLAUDE.md for the column list.
    Pass `path` if the CSV is not in data/processed/ (e.g. uploaded to a chat)."""
    return pd.read_csv(path or PROCESSED / "route801_events.csv.gz", parse_dates=["service_day", "arrival"])


def load_stops():
    """Stop IDs in route order (stop_order 1 = first stop)."""
    return pd.read_csv(PROCESSED / "stops.csv", dtype={"stop_id": str})


def window(df):
    """Analysis window: weekdays 7:00-17:59, when buses are scheduled every 10 minutes."""
    return df[df.in_window]


# ---------------------------------------------------------------- style
# Colours: use them in this order for categories; never pick random colours.
COLORS = {
    "blue": "#2a78d6",     # main series
    "orange": "#eb6834",   # second series / highlight
    "aqua": "#1baf7a",     # third series
    "text": "#0b0b0b",
    "muted": "#52514e",
    "grid": "#e4e3df",
}
SERIES = [COLORS["blue"], COLORS["orange"], COLORS["aqua"]]
# One-hue blue scale for heatmaps (light = low, dark = high).
SEQ_CMAP = LinearSegmentedColormap.from_list(
    "seq_blue", ["#cde2fb", "#9ec5f4", "#6da7ec", "#3987e5", "#256abf", "#184f95", "#0d366b"])

plt.rcParams.update({
    "figure.dpi": 110,
    "savefig.dpi": 200,
    "figure.facecolor": "white",
    "axes.facecolor": "white",
    "font.family": "DejaVu Sans",
    "font.size": 13,
    "axes.titlesize": 16,
    "axes.titleweight": "bold",
    "axes.titlelocation": "left",
    "axes.titlepad": 12,
    "axes.labelsize": 13,
    "axes.labelcolor": COLORS["text"],
    "axes.edgecolor": COLORS["muted"],
    "axes.linewidth": 0.8,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "axes.axisbelow": True,
    "grid.color": COLORS["grid"],
    "grid.linewidth": 0.8,
    "xtick.color": COLORS["muted"],
```

```
    "ytick.color": COLORS["muted"],
    "xtick.labelsize": 12,
    "ytick.labelsize": 12,
    "legend.frameon": False,
    "legend.fontsize": 12,
    "lines.linewidth": 2,
    "axes.prop_cycle": plt.cycler(color=SERIES),
})


def new_fig(width=10, height=5.6):
    """16:9 figure that fits a slide and a page. Returns (fig, ax)."""
    return plt.subplots(figsize=(width, height))


def finish(ax, title, xlabel, ylabel, note=None):
    """Title, axis labels (always with units) and an optional source/note line under the plot."""
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    if note:
        ax.annotate(note, xy=(0, -0.18), xycoords="axes fraction", fontsize=10.5,
                    color=COLORS["muted"], ha="left", va="top")


def save(fig, name, folder=None):
    """Save to figures/<name>.png (200 dpi), or to `folder` if given."""
    out = Path(folder) if folder else FIGURES
    out.mkdir(parents=True, exist_ok=True)
    path = out / f"{name}.png"
    fig.savefig(path, bbox_inches="tight", facecolor="white")
    return path


SOURCE_NOTE = "Source: CapMetro APC data, Route 801 southbound, Jul–Dec 2021"
```